

Physics Lab

```
library(readxl)

data <- read_excel("Capstone Data.xlsx")

dim(data)

## [1] 987 28

runs_to_remove <- c(5, 6, 7, 12, 13, 14)

pattern <- paste0("Run #(", paste(runs_to_remove, collapse = "|"), ")$")

data <- data[, !grepl(pattern, names(data))]

names(data)

## [1] "Relative Intensity (%)" "Angle (rad)" "Run #1"
## [3] "Relative Intensity (%)" "Angle (rad)" "Run #2"
## [5] "Relative Intensity (%)" "Angle (rad)" "Run #3"
## [7] "Relative Intensity (%)" "Angle (rad)" "Run #4"
## [9] "Relative Intensity (%)" "Angle (rad)" "Run #8"
## [11] "Relative Intensity (%)" "Angle (rad)" "Run #9"
## [13] "Relative Intensity (%)" "Angle (rad)" "Run #10"
## [15] "Relative Intensity (%)" "Angle (rad)" "Run #11"

library(tidyr)
library(dplyr)

##
## Attaching package: 'dplyr'

## The following objects are masked from 'package:stats':
##
## filter, lag

## The following objects are masked from 'package:base':
##
## intersect, setdiff, setequal, union

data_long <- data %>%
  pivot_longer(
    cols = everything(),
    names_to = c("measurement", "run"),
```

```

names_pattern = "^(.*) Run #(\\d+)$",
values_to = "value"
) %>%
mutate(run = as.integer(run)) %>%
pivot_wider(
  names_from = measurement,
  values_from = value
)

```

```

## Warning: Values from 'value' are not uniquely identified; output will contain list-cols.
## * Use 'values_fn = list' to suppress this warning.
## * Use 'values_fn = {summary_fun}' to summarise duplicates.
## * Use the following dplyr code to identify duplicates.
## {data} |>
## dplyr::summarise(n = dplyr::n(), .by = c(run, measurement)) |>
## dplyr::filter(n > 1L)

```

```
head(data_long)
```

```

## # A tibble: 6 x 3
##   run 'Relative Intensity (%)' 'Angle (rad)'
##   <int> <list>                <list>
## 1     1 <dbl [987]>                <dbl [987]>
## 2     2 <dbl [987]>                <dbl [987]>
## 3     3 <dbl [987]>                <dbl [987]>
## 4     4 <dbl [987]>                <dbl [987]>
## 5     8 <dbl [987]>                <dbl [987]>
## 6     9 <dbl [987]>                <dbl [987]>

```

```
data_long$run
```

```
## [1] 1 2 3 4 8 9 10 11
```

```

data_long <- data_long %>%
  unnest(c('Relative Intensity (%)', 'Angle (rad)'))

```

```
dim(data_long)
```

```
## [1] 7896 3
```

```
head(data_long)
```

```

## # A tibble: 6 x 3
##   run 'Relative Intensity (%)' 'Angle (rad)'
##   <int> <dbl>                <dbl>
## 1     1 0.1                0
## 2     1 0.11               0
## 3     1 0.04                0
## 4     1 0.09                0
## 5     1 0.09                0
## 6     1 0.07                0

```

```
data_long <- data_long %>%
  rename(
    intensity = `Relative Intensity (%)`,
    angle = `Angle (rad)`
  )

head(data_long)
```

```
## # A tibble: 6 x 3
##   run intensity angle
##   <int>      <dbl> <dbl>
## 1     1      0.1    0
## 2     1     0.11    0
## 3     1     0.04    0
## 4     1     0.09    0
## 5     1     0.09    0
## 6     1     0.07    0
```

RUN 1 V=4

```
run1 <- data_long %>%
  filter(run == 1)

run1_peak <- run1 %>%
  filter(angle > -25, angle < -8)

peak1 <- run1_peak %>%
  slice_max(intensity, n = 1)

peak1
```

```
## # A tibble: 1 x 3
##   run intensity angle
##   <int>      <dbl> <dbl>
## 1     1     35.8 -15.9
```

```
peak1$angle
```

```
## [1] -15.882
```

RUN2 V= 5V

```
run2 <- data_long %>%
  filter(run == 2)

run2_peak <- run2 %>%
  filter(angle > -25, angle < -8)
```

```
peak2 <- run2_peak %>%
  slice_max(intensity, n = 1)
```

```
peak2
```

```
## # A tibble: 1 x 3
##   run intensity angle
##   <int>      <dbl> <dbl>
## 1     2      51.5 -16.1
```

```
peak2$angle
```

```
## [1] -16.074
```

RUN 3 V=6V

```
run3 <- data_long %>%
  filter(run == 3)
```

```
run3_peak <- run3 %>%
  filter(angle > -25, angle < -8)
```

```
peak3 <- run3_peak %>%
  slice_max(intensity, n = 1)
```

```
peak3
```

```
## # A tibble: 1 x 3
##   run intensity angle
##   <int>      <dbl> <dbl>
## 1     3      73.0 -15.7
```

```
peak3$angle
```

```
## [1] -15.673
```

RUN 4 V=7V

```
run4 <- data_long %>%
  filter(run == 4)
```

```
run4_peak <- run4 %>%
  filter(angle > -25, angle < -8)
```

```
peak4 <- run4_peak %>%
  slice_max(intensity, n = 1)
```

```
peak4
```

```
## # A tibble: 2 x 3
##   run intensity angle
##   <int>      <dbl> <dbl>
## 1     4      92.5 -15.7
## 2     4      92.5 -15.7
```

```
peak4$angle
```

```
## [1] -15.656 -15.691
```

RUN 8 V=4V

```
run8 <- data_long %>%
  filter(run == 8)

run8_peak <- run8 %>%
  filter(angle > -25, angle < -8)

peak8 <- run8_peak %>%
  slice_max(intensity, n = 1)

peak8
```

```
## # A tibble: 1 x 3
##   run intensity angle
##   <int>      <dbl> <dbl>
## 1     8      36.5 -16.0
```

```
peak8$angle
```

```
## [1] -16.04
```

RUN 9 V=5V

```
run9 <- data_long %>%
  filter(run == 9)

run9_peak <- run9 %>%
  filter(angle > -25, angle < -8)

peak9 <- run9_peak %>%
  slice_max(intensity, n = 1)

peak9
```

```
## # A tibble: 2 x 3
##   run intensity angle
##   <int>      <dbl> <dbl>
## 1     9      52.5 -15.7
## 2     9      52.5 -15.8
```

```
peak9$angle
```

```
## [1] -15.743 -15.795
```

RUN 10 V=6V

```
run10 <- data_long %>%
  filter(run == 10)

run10_peak <- run10 %>%
  filter(angle > -25, angle < -8)

peak10 <- run10_peak %>%
  slice_max(intensity, n = 1)

peak10
```

```
## # A tibble: 1 x 3
##   run intensity angle
##   <int>      <dbl> <dbl>
## 1    10      69.4 -16.0
```

```
peak10$angle
```

```
## [1] -16.04
```

RUN 11 V=7V

```
run11 <- data_long %>%
  filter(run == 11)

run11_peak <- run11 %>%
  filter(angle > -25, angle < -8)

peak11 <- run11_peak %>%
  slice_max(intensity, n = 1)

peak11
```

```
## # A tibble: 1 x 3
##   run intensity angle
##   <int>      <dbl> <dbl>
## 1    11      91.5 -15.8
```

```
peak11$angle
```

```
## [1] -15.76
```

```

peak_table <- data_long %>%
  filter(angle > -25, angle < -8) %>%
  group_by(run) %>%
  slice_max(
    order_by = intensity,
    n = 1,
    with_ties = FALSE
  ) %>%
  ungroup() %>%
  select(
    run,
    peak_angle = angle,
    peak_intensity = intensity
  )

```

peak_table

```

## # A tibble: 8 x 3
##   run peak_angle peak_intensity
##   <int>      <dbl>         <dbl>
## 1     1      -15.9           35.8
## 2     2      -16.1           51.5
## 3     3      -15.7           73.0
## 4     4      -15.7           92.5
## 5     8      -16.0           36.5
## 6     9      -15.7           52.5
## 7    10      -16.0           69.4
## 8    11      -15.8           91.5

```

```

library(ggplot2)
library(dplyr)

plot_data <- data_long %>%
  mutate(
    measurement = if_else(
      run %in% c(8, 9, 10, 11),
      "Repeat",
      "First"
    ),
    run = factor(run)
  )

run_colors <- c(
  "1" = "#1b9e77",
  "2" = "#d95f02",
  "3" = "#7570b3",
  "4" = "#e7298a",
  "8" = "#1b9e77",
  "9" = "#d95f02",
  "10" = "#7570b3",
  "11" = "#e7298a"
)

```

```

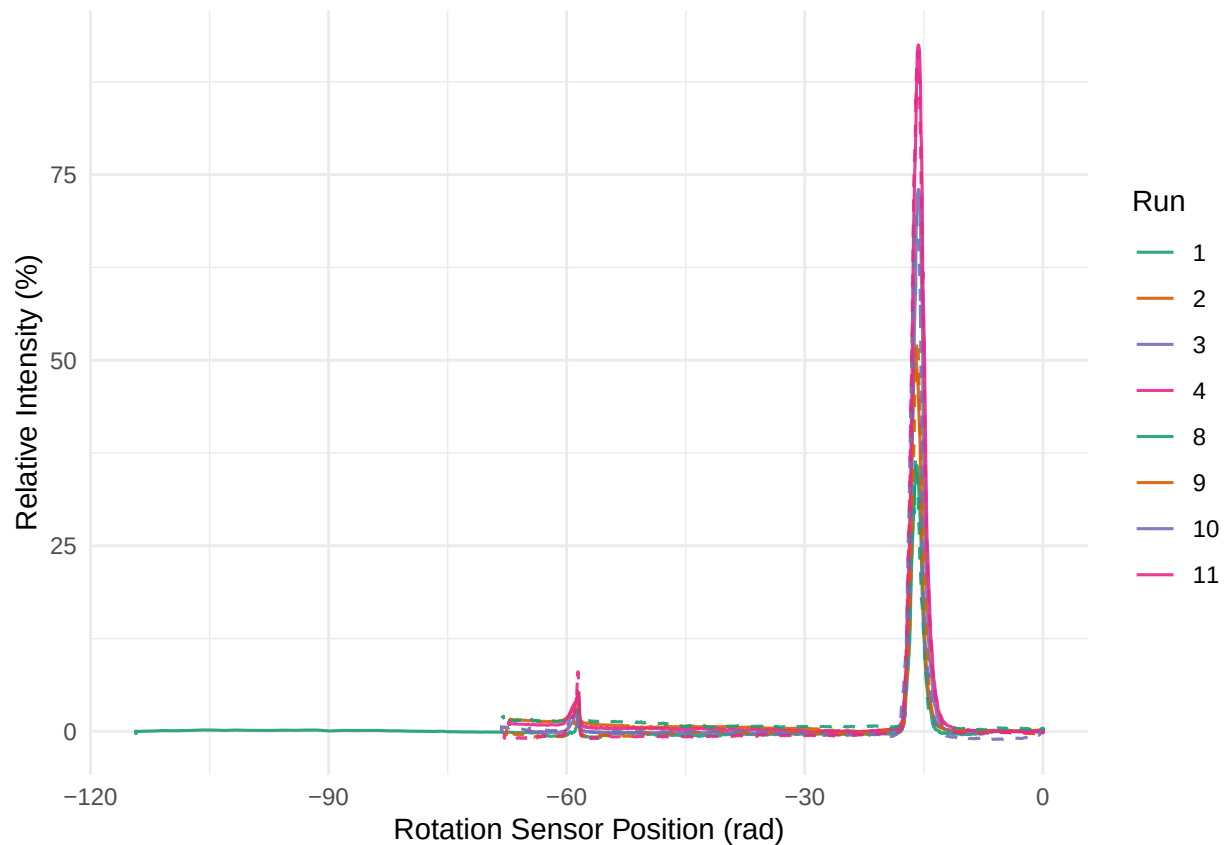
ggplot(
  plot_data,
  aes(
    x = angle,
    y = intensity,
    color = run,
    linetype = measurement
  )
) +
  geom_line(
    linewidth = 0.55,
    alpha = 0.9
  ) +
  scale_color_manual(
    values = run_colors,
    name = "Run"
  ) +
  scale_linetype_manual(
    values = c(
      "First" = "solid",
      "Repeat" = "dashed"
    ),
    guide = "none"
  ) +
  labs(
    x = "Rotation Sensor Position (rad)",
    y = "Relative Intensity (%)"
  ) +
  theme_minimal()

```

```

## Warning: Removed 827 rows containing missing values or values outside the scale range
## ('geom_line()').

```



```
data <- data.frame(
  temperature = c(1660, 1860, 2030, 2190),
  lambda_max = c(1000, 980, 960, 910)
)
```

```
data$inv_T <- 1 / data$temperature
```

```
fit <- lm(lambda_max ~ inv_T, data = data)
```

```
summary(fit)
```

```
##
## Call:
## lm(formula = lambda_max ~ inv_T, data = data)
##
## Residuals:
##      1      2      3      4
## -8.433  8.716 14.538 -14.822
##
## Coefficients:
##              Estimate Std. Error t value Pr(>|t|)
## (Intercept)    662.95      82.03   8.082  0.0150 *
## inv_T         573506.60 156197.12   3.672  0.0668 .
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
##
```

```
## Residual standard error: 17 on 2 degrees of freedom
## Multiple R-squared:  0.8708, Adjusted R-squared:  0.8062
## F-statistic: 13.48 on 1 and 2 DF,  p-value: 0.06683
```

```
library(ggplot2)
library(dplyr)

plot_data <- data_long %>%
  filter(angle >= -20, angle <= -10) %>%
  mutate(
    run = as.character(run),
    measurement = if_else(
      run %in% c("8", "9", "10", "11"),
      "Repeat",
      "First"
    )
  )

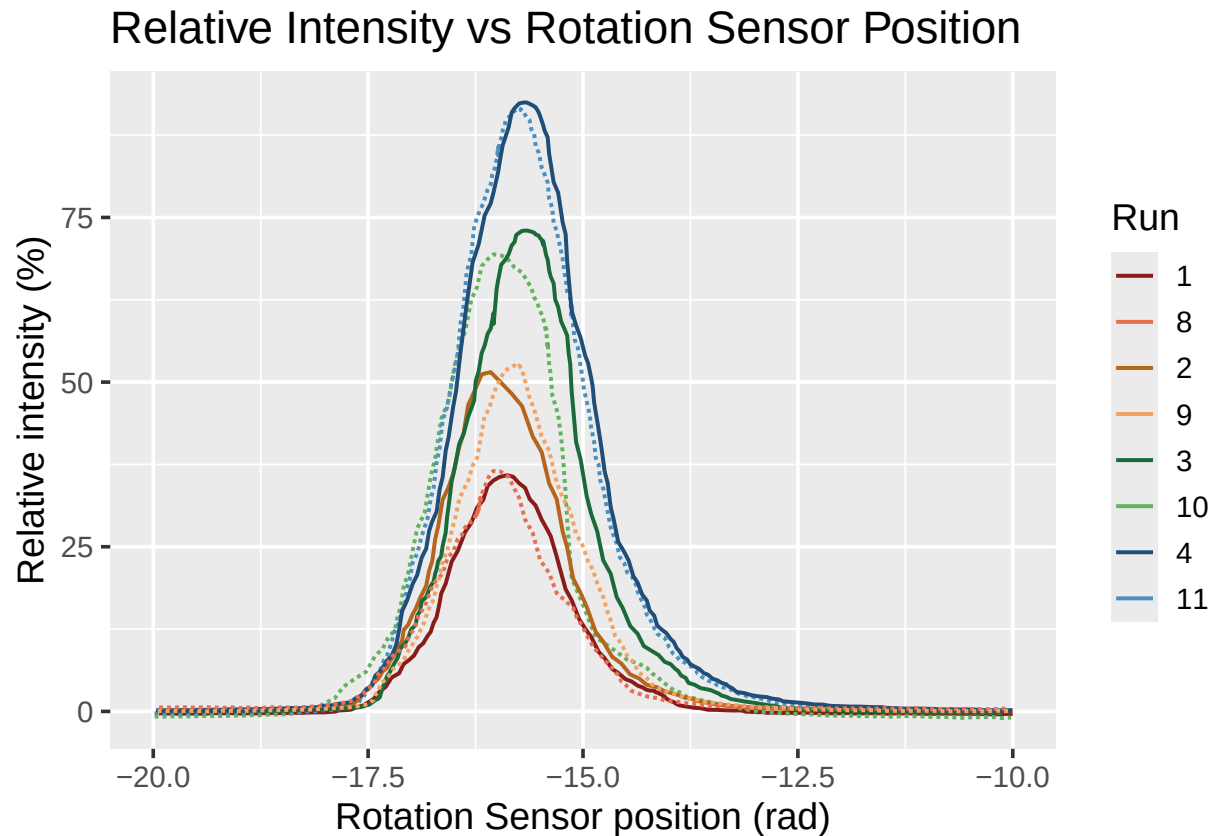
run_colors <- c(
  "1" = "#8B1A1A",
  "8" = "#E76F51",
  "2" = "#B5651D",
  "9" = "#F4A261",
  "3" = "#1B6B3A",
  "10" = "#63B35D",
  "4" = "#1F4E79",
  "11" = "#4A90C2"
)

ggplot(
  plot_data,
  aes(
    x = angle,
    y = intensity,
    color = run,
    linetype = measurement
  )
) +
  geom_line(
    linewidth = 0.7
  ) +
  scale_color_manual(
    values = run_colors,
    breaks = c("1", "8", "2", "9", "3", "10", "4", "11"),
    name = "Run"
  ) +
  scale_linetype_manual(
    values = c(
      "First" = "solid",
      "Repeat" = "11"
    ),
    guide = "none"
  ) +
  scale_x_continuous(
```

```

limits = c(-20, -10),
breaks = c(-20, -17.5, -15, -12.5, -10)
) +
labs(
  x = "Rotation Sensor position (rad)",
  y = "Relative intensity (%)",
  title = "Relative Intensity vs Rotation Sensor Position"
) +
theme_gray(base_size = 14)

```



```

paired_peaks <- tibble(
  voltage = c(4, 5, 6, 7),

  run1 = c(1, 2, 3, 4),
  run2 = c(8, 9, 10, 11),

  angle1 = c(
    -15.882,
    -16.074,
    -15.673,
    -15.656
  ),

  angle2 = c(
    -16.040,
    -15.743,

```

```

    -16.040,
    -15.760
  )
)

paired_peaks <- paired_peaks %>%
  mutate(
    mean_angle = (angle1 + angle2) / 2,
    sd_angle = abs(angle1 - angle2) / sqrt(2),
    error_mean_angle = sd_angle / sqrt(2)
  )

final_data <- paired_peaks %>%
  mutate(
    temperature = c(1660, 1860, 2030, 2190),
    lambda_max = c(1000, 980, 960, 910),
    inv_T = 1 / temperature
  )

final_data

## # A tibble: 4 x 11
##   voltage run1 run2 angle1 angle2 mean_angle sd_angle error_mean_angle
##   <dbl> <dbl> <dbl> <dbl> <dbl> <dbl> <dbl> <dbl>
## 1     4     1     8  -15.9  -16.0  -16.0  0.112  0.0790
## 2     5     2     9  -16.1  -15.7  -15.9  0.234  0.166
## 3     6     3    10  -15.7  -16.0  -15.9  0.260  0.183
## 4     7     4    11  -15.7  -15.8  -15.7  0.0735 0.0520
## # i 3 more variables: temperature <dbl>, lambda_max <dbl>, inv_T <dbl>

fit <- lm(lambda_max ~ inv_T, data = final_data)

summary(fit)

##
## Call:
## lm(formula = lambda_max ~ inv_T, data = final_data)
##
## Residuals:
##      1      2      3      4
## -8.433  8.716 14.538 -14.822
##
## Coefficients:
##              Estimate Std. Error t value Pr(>|t|)
## (Intercept)    662.95      82.03   8.082  0.0150 *
## inv_T        573506.60 156197.12   3.672  0.0668 .
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
##
## Residual standard error: 17 on 2 degrees of freedom
## Multiple R-squared:  0.8708, Adjusted R-squared:  0.8062
## F-statistic: 13.48 on 1 and 2 DF, p-value: 0.06683

```

```
slope <- coef(fit)[2]
intercept <- coef(fit)[1]
r2 <- summary(fit)$r.squared
```

```
data <- data.frame(
  temperature = c(1660, 1860, 2030, 2190),
  lambda_max = c(1000, 980, 960, 910)
)
```

```
data$inv_T <- 1 / data$temperature
```

```
fit <- lm(lambda_max ~ inv_T, data = data)
```

```
summary(fit)
```

```
##
## Call:
## lm(formula = lambda_max ~ inv_T, data = data)
##
## Residuals:
##      1      2      3      4
## -8.433  8.716 14.538 -14.822
##
## Coefficients:
##              Estimate Std. Error t value Pr(>|t|)
## (Intercept)    662.95      82.03   8.082  0.0150 *
## inv_T       573506.60 156197.12   3.672  0.0668 .
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
##
## Residual standard error: 17 on 2 degrees of freedom
## Multiple R-squared:  0.8708, Adjusted R-squared:  0.8062
## F-statistic: 13.48 on 1 and 2 DF, p-value: 0.06683
```

```
library(ggplot2)
```

```
ggplot(data, aes(x = inv_T, y = lambda_max)) +
  geom_point(size = 3) +
  geom_smooth(
    method = "lm",
    se = FALSE
  ) +
  labs(
    x = expression("Inverse temperature, " ~ 1/T ~ (K^{-1})),
    y = expression("Peak wavelength, " ~ lambda[max] ~ (nm)),
    title = expression(lambda[max] ~ "vs" ~ 1/T)
  ) +
  theme_minimal()
```

```
## 'geom_smooth()' using formula = 'y ~ x'
```

